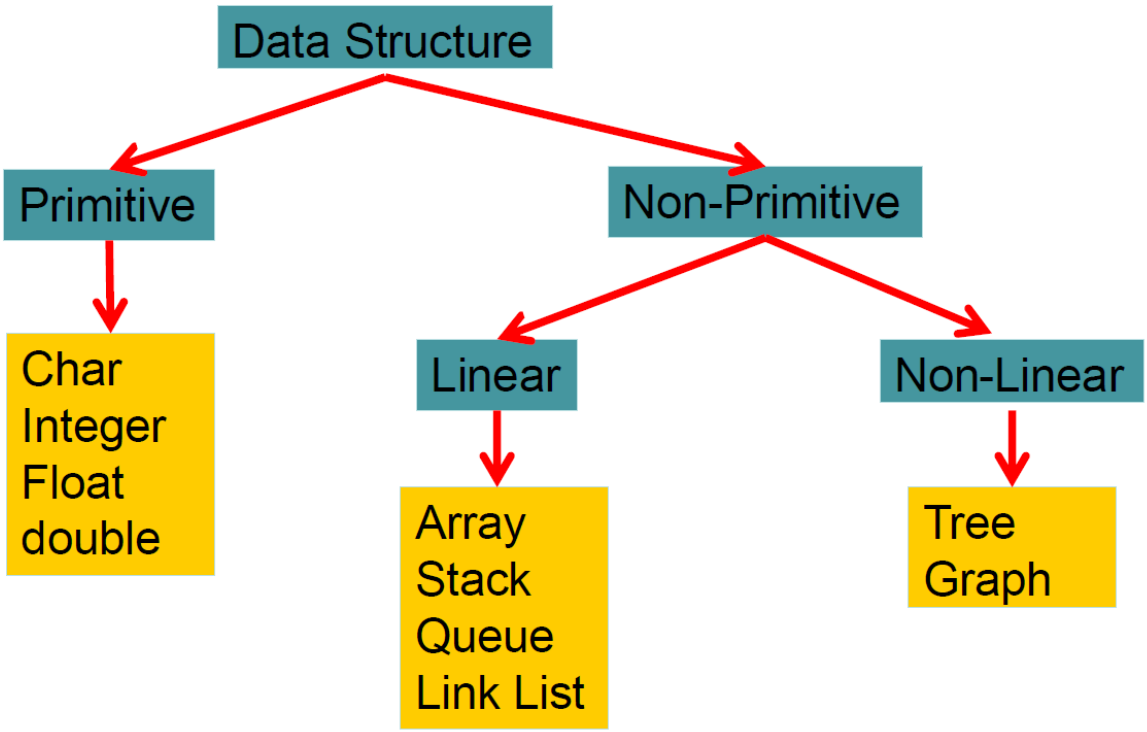
Slide -1

 # What is Data Structure?

A data structure is a way to organize, store, and manage data in a computer so it can be used efficiently. Think of it as a container that holds data, with rules on how to organize and access it, making tasks like searching, sorting, or updating data faster and more effective. Some of the more commonly used data structures include lists, arrays, stacks, queues, heaps, trees, and graphs.

#Types of Data Structure:



1. Linear Data Structures

- Data is arranged in a sequence or line, and each element is connected to its previous and next elements. Allowing direct access to only one element.
- Examples:
  - Array: A collection of elements stored at contiguous memory locations.
  - Linked List: A series of nodes, each pointing to the next.
  - Stack: Follows Last-In-First-Out (LIFO)—like stacking plates.
  - Queue: Follows First-In-First-Out (FIFO)—like a line at a ticket counter.

2. Non-Linear Data Structures

- Data is not arranged in a sequence, and elements may be connected in complex ways.
- Examples:
  - Tree: A hierarchical structure, like a family tree.
  - Graph: A set of nodes connected by edges, useful for representing networks.
  - Heap: A special tree-based structure used for priority-based tasks.

 # What is Abstract Data Type (ADT)?

An Abstract Data Type (ADT) describes how data is viewed and the operations allowed, without focusing on implementation details. It defines a type and a set of operations for that type. ADTs hide internal implementation details from the user.

Example

An ADT for a list of integers might include operations like:

1. Insert a new integer at a specific position.
2. Check if the list is empty (returns true/false).
3. Get the total number of integers.
4. Find the maximum integer.
5. Update the list.
6. Sort the list.

 # List vs Array:

List and arrays are two built-in data structures that can be used to organize data, or to create other data

List	Array
A dynamic collection of elements where each item is linked to the next (in case of linked lists).	A fixed-size collection of elements stored in contiguous memory.
Size can grow or shrink dynamically.	Size is fixed at the time of creation.
Non-contiguous memory allocation (for linked lists).	Contiguous memory allocation.
Slower for accessing elements (sequential access).	Faster for accessing elements (index-based random access).
Efficient, as it doesn't require shifting elements (in linked lists).	Relatively slow, as elements might need to be shifted.
Can store elements of different data types (e.g., in Python lists).	Usually stores elements of the same data type.
More flexible due to dynamic resizing and heterogeneous data storage.	Less flexible due to fixed size and homogeneous data type.

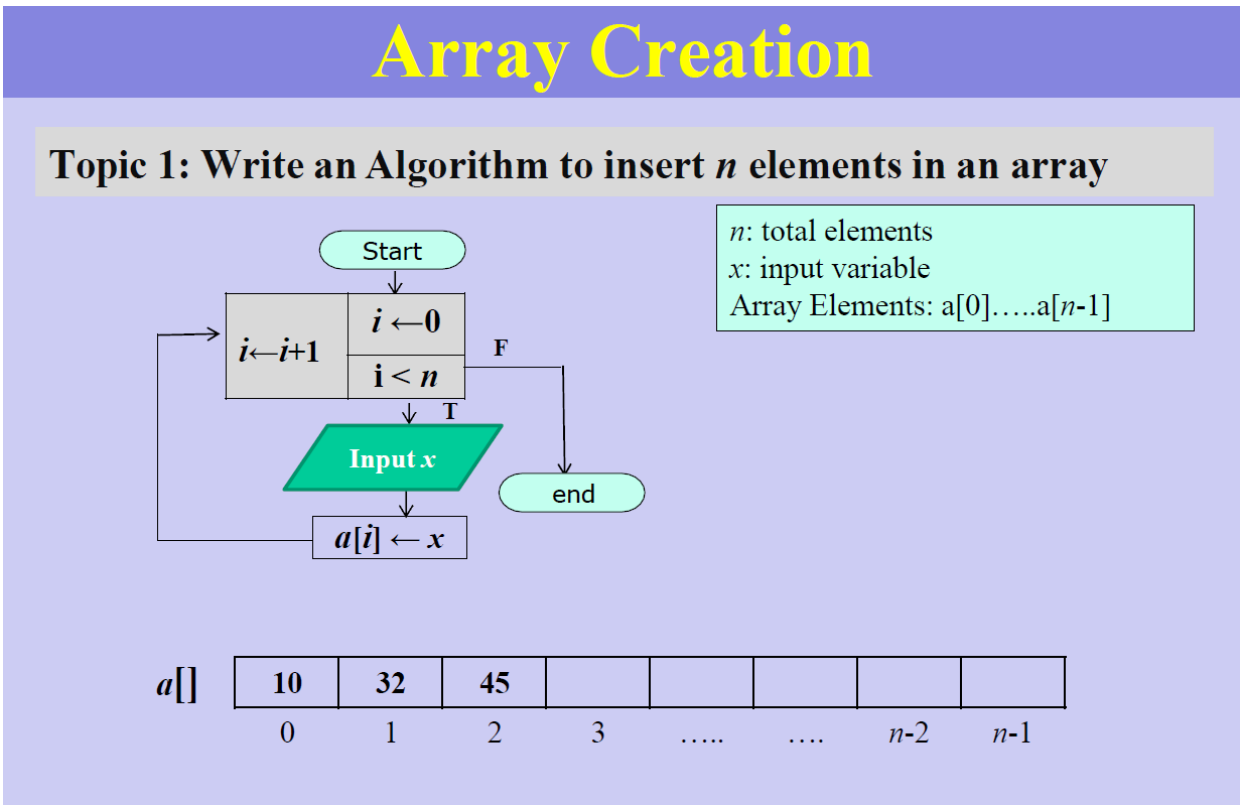
Slide -2

Array

A collection of elements stored in contiguous memory locations. You can access elements using their index (position).

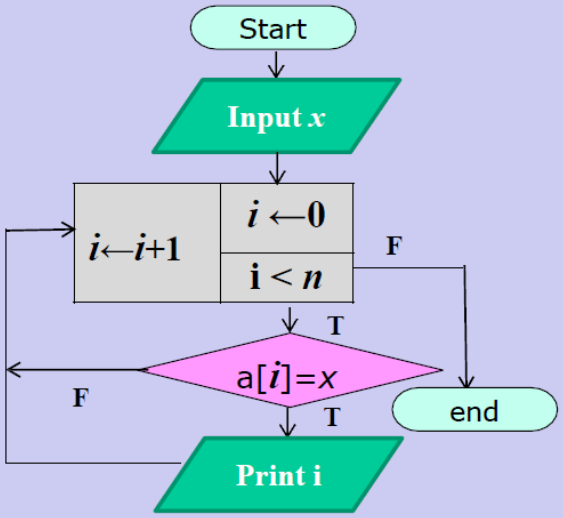
A few types:

- Pointer-based arrays (C-like)
- Arrays as objects (C++)



Array Searching

Topic 2: Write an Algorithm to search an element(s) in an array

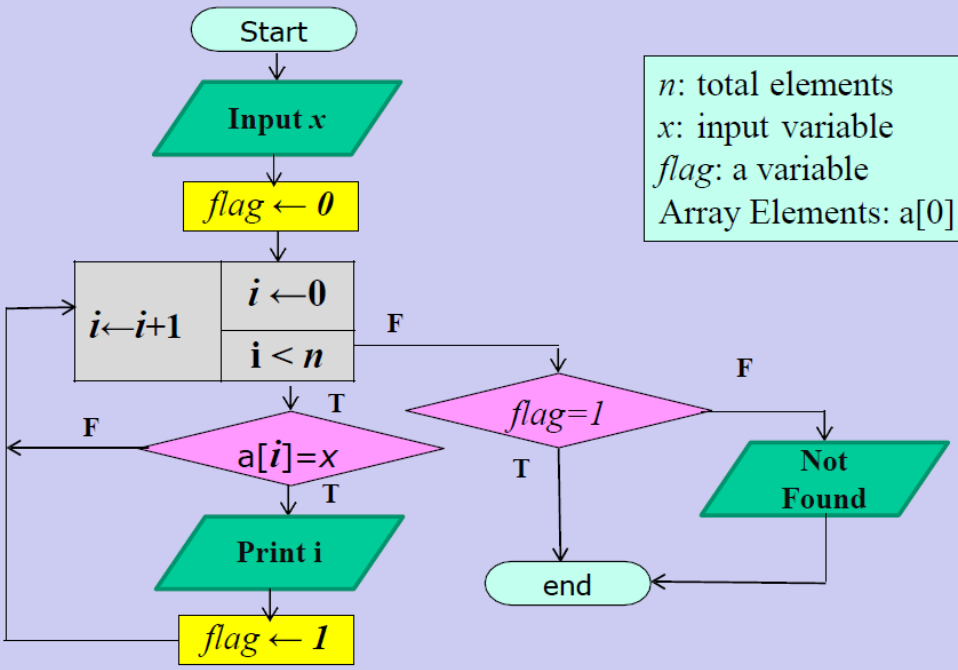


n: total elements
x: input variable
Array Elements: a[0].....a[*n*-1]

a[]	10	32	45					
	0	1	2	3	.....	....	*n*-2	*n*-1

Array Searching (cont'd)

Topic 3: Modifications of previous algorithm

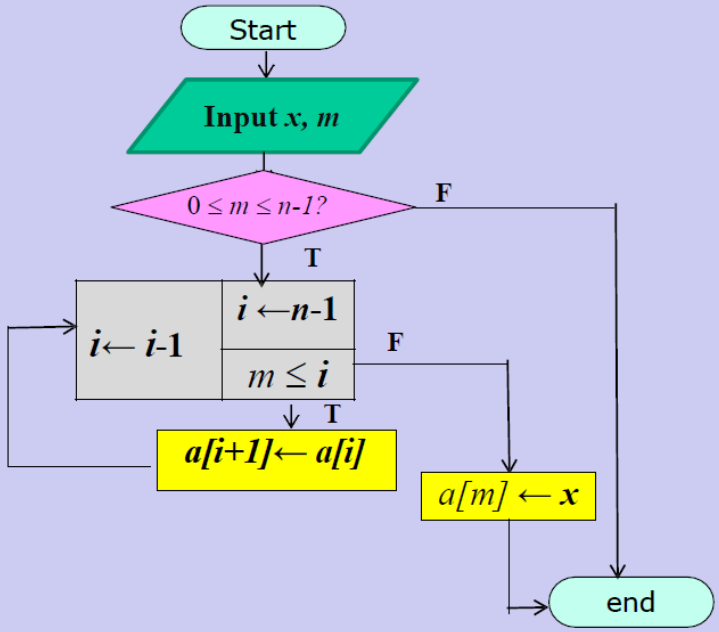


n: total elements
x: input variable
flag: a variable
Array Elements: a[0].....a[*n*-1]

Complexity
O(1) for best-case
O(n) for worst-case

Array Insertion

Topic 4: Insert a new element at index *m*.



n: total elements
m: index $0 \le m \le n-1$
x: input variable for new data
Array Elements: *a*[0]…..*a*[*n-1*]

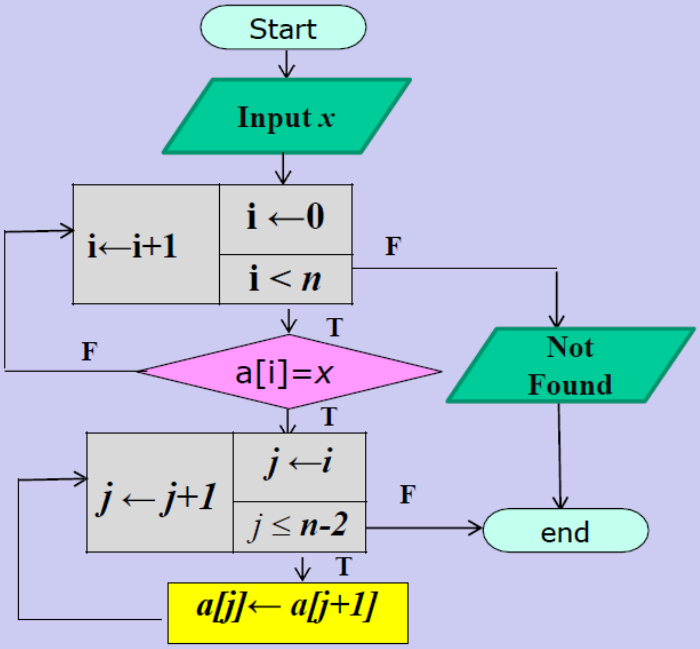
Shifting Required

all elements from index ***m*** to ***(n-1)*** are needed to be shifted to index ***(m+1)*** to ***n*** respectively. Total ***(n-m)*** elements to be shifted.

a[]	10	32	45				
	0	1	….	*m*	…	*n-1*	*n*

Array Insertion (cont'd)

Topic 4: Delete a specific element.



n: total elements
m: index $0 \le m \le n$
x: input variable to be deleted
Array Elements: *a[0]*…..*a[n-1]*

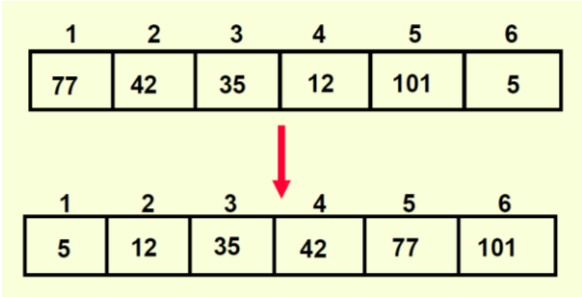
Shifting Required

all elements from index ***(m+1)*** to ***(n-1)*** are needed to be shifted from index ***m*** to ***(n-2)*** respectively. Total ***(n-m-1)*** elements to be shifted.

a[]	10	32	45				
	0	1	….	*m*	…	*n-1*	*n*

Sorting (Comparison Reduce)

Definition: Sorting means arranging data in a specific order (ascending or descending).



Why reduces comparisons?

- To make sorting faster and more efficient
- Less comparisons = less time = better performance
-

Short Theory: Reducing Comparisons in Sorting
In sorting, comparisons are used to check the order between elements. The more comparisons, the slower the sorting becomes. To make sorting faster, we should reduce unnecessary comparisons.

Basic sorting algorithms like **Bubble Sort** or **Selection Sort** use many comparisons ($O(n^2)$), which is not efficient for large data.

To reduce comparisons, we use advanced algorithms like:

- **Merge Sort**: Uses divide and conquer, reduces comparisons to $O(n \log n)$.
- **Quick Sort**: Uses smart pivot selection to sort quickly.
- **Heap Sort**: Builds a heap to minimize comparisons during sorting.

By using these methods, we can sort large datasets efficiently with fewer comparisons.

What is $O(n^2)$?
$O(n^2)$ is part of something called **Big O Notation** — it shows how **fast or slow** an algorithm is when the input gets **bigger**.

Meaning of $O(n^2)$:

- It means the **time (or steps)** an algorithm takes is **proportional to the square of the input size**.
- If input = n, then steps = $n \times n = n^2$.

So if you have:

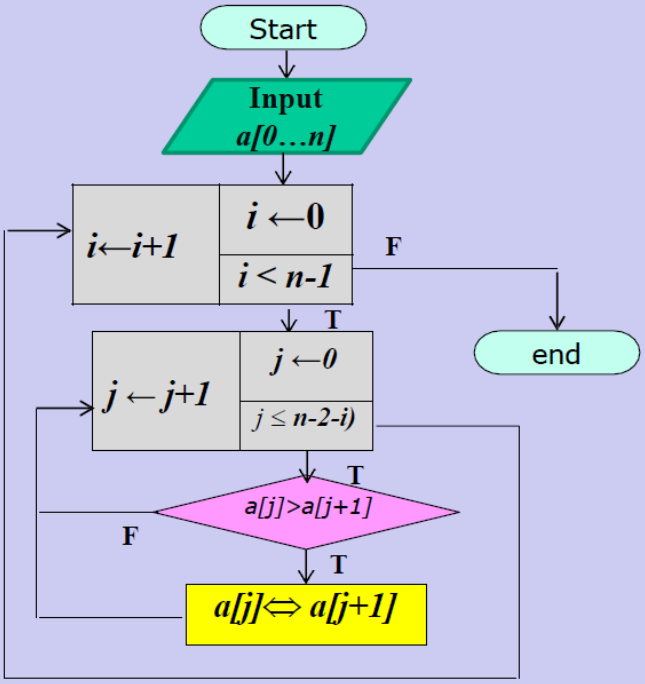
- 10 items → about 100 steps
- 100 items → about 10,000 steps

That's **really slow** for big inputs!

Example: Bubble Sort has $O(n^2)$ time complexity. If you give it 1,000 items, it might compare them around **1,000,000 times**!

Bubble Sort

Topic 5: Wrte an algorithm to sort an array using Bubble Sort.



n: total elements
Array Elements: a[0]…..a[n-1]

```
#include  <stdio.h>
#include  <stdlib.h>
int main()
{
   int a[6]={10,3,41,12,77,21};
   int n=6;
   int i,j,t;
   for(i=0;i<n-1;i++)
    for(j=0;j<=n-2-i;j++)
    if(a[j]>a[j+1])
   {
       t=a[j];a[j]=a[j+1];a[j+1]=t;
   }
   for(i=0;i<n;i++)
    printf("%d ",a[i]);
    return 0;
}
```

Corresponding C program

Sparse Matrix

Definition: Sparse Matrix

A Sparse Matrix is a matrix where most of the elements are zero.

In other words, it has very few non-zero elements compared to the total number of elements.

Why use Sparse Matrix?

- To save memory (don't store the zeros)
- To speed up computation
- Useful in scientific computing, graph algorithms, machine learning, etc.

📊 **Example:**

A 3×3 matrix:

```
0  0  5
0  0  0
0  2  0
```

Total elements = 9

Non-zero elements = 2

Since most values are zero → this is a **sparse matrix** ✅

Triplet Representation of Sparse Matrix

Triplet Representation is a method to store sparse matrices efficiently by only saving the non-zero elements and their positions. Each triplet stores the row index, column index, and value of the non-zero element, reducing memory usage.

Why Use Triplet Representation?

- Memory-efficient: Only stores non-zero values.
- Faster processing: Helps in reducing time for operations like addition, multiplication, etc.

Example:

Consider the following sparse matrix:

```
0 0 5
0 0 0
0 2 0
```

We can represent this matrix using **Triplet Representation** like this:

Row	Column	Value
0	2	5
2	1	2

Slide -3

What is a Stack?

A Stack is a linear data structure that follows the LIFO (Last In, First Out) principle. This means:

- Last In (the last item added) is the first out (the first to be removed).

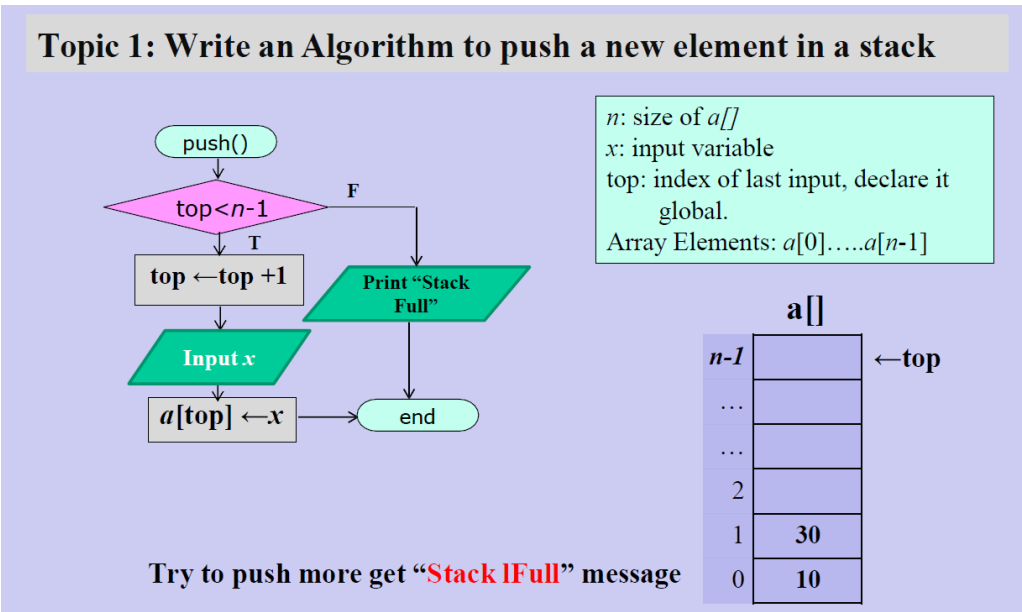
Think of a stack of plates: you add plates on top, and to remove one, you take the top plate first.

Push Operation

Push is the operation used to add an element to the top of the stack. When you push an element, it is placed on the top of the stack, and the stack size increases by 1.

Push Algorithm:

1. Check if full: If yes, show overflow error.
2. Move top pointer to the next position.
3. Insert element at the top.
4. Done.

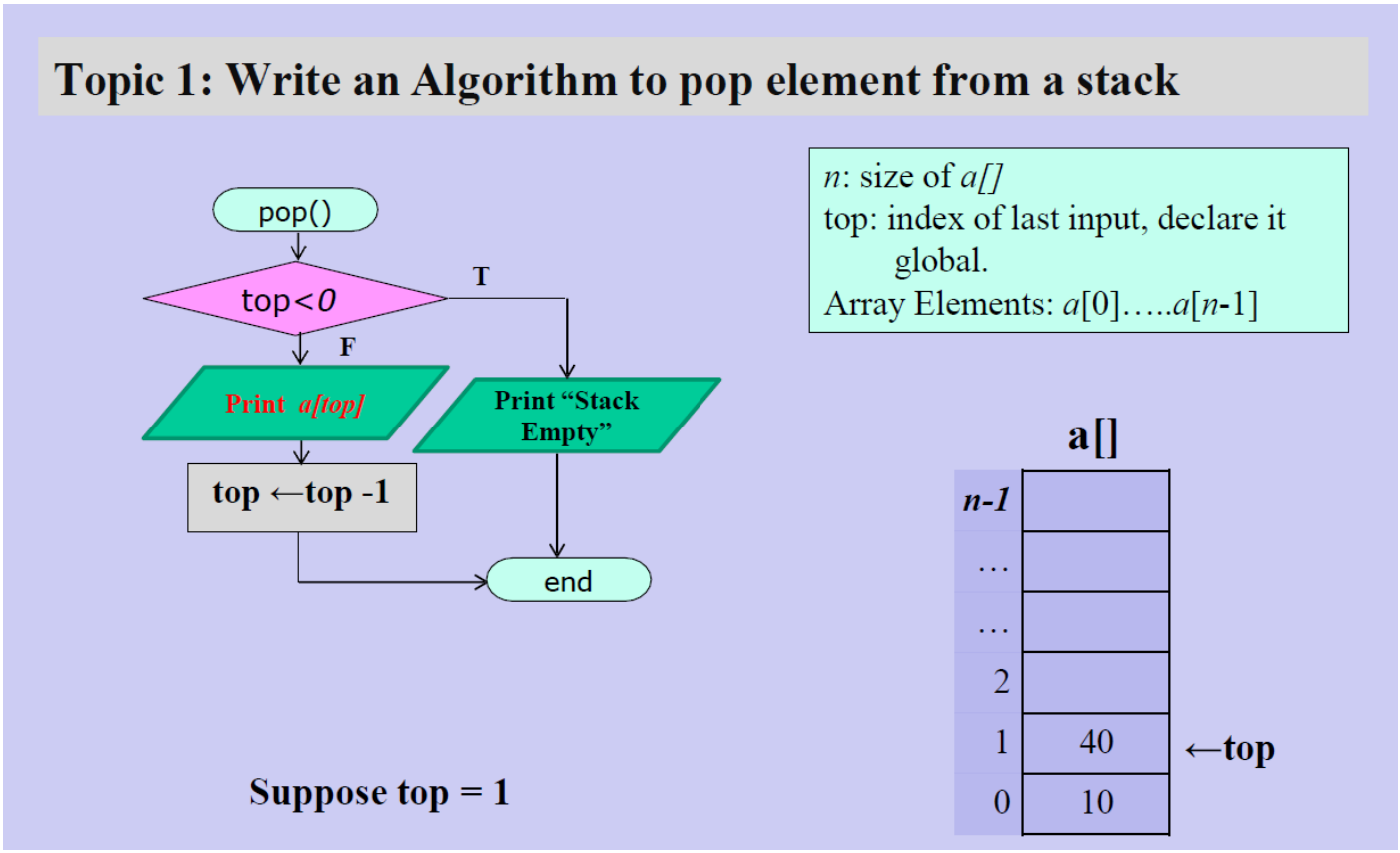


Pop Operation

Pop is the operation used to remove the top element from the stack. When you pop an element, it is removed from the top of the stack, and the stack size decreases by 1.

Pop Algorithm:

1. Check if empty: If yes, show underflow error.
2. Remove top element.
3. Move top pointer to the previous position.
4. Done.



What is Polish Notation?

Polish Notation is a way of writing expressions **without using brackets** and **based on the position of operators.**

Types Of Polish Notation:

1. Infix
2. prefix
3. Postfix

What They Are

Notation	**Definition**	**Example (2 + 3 * 4)**
Infix	Operator is **between** operands	2 + 3 * 4
Prefix	Operator is **before** the operands (Polish)	+2* 34
Postfix	Operator is **after** the operands (Reverse Polish)	234 * +

Key Points

- Infix: Commonly used by humans, needs parentheses for priority.
- Prefix: No brackets needed, operator comes first.
- Postfix: No brackets needed, operator comes last, easy for stacks.

✅ Extra Examples

Infix	Prefix	Postfix
A + B	+ A B	A B +
A + B * C	+ A * B C	A B C * +
(A + B) * C	* + A B C	A B + C *

-By MD.Pranto